

REPORTE FINAL DE PROYECTO

Optimización de Redes Convolucionales (TinyVGG)

Actividad 3 | Maestría en Ciencia de Datos

1. INTRODUCCIÓN Y OBJETIVOS

Marco Teórico: Redes Neuronales Convolucionales (CNN)

Las **Redes Neuronales Convolucionales (CNN)** son un tipo especializado de arquitectura de Deep Learning diseñada para procesar datos con estructura de rejilla, como las imágenes. A diferencia de las redes neuronales densas tradicionales, las CNN utilizan operaciones matemáticas llamadas "convoluciones" que emplean filtros (o kernels) para escanear la imagen y extraer patrones visuales de manera automática.

En tareas de **clasificación de imágenes**, las CNN funcionan de manera jerárquica: las primeras capas detectan características simples como líneas, bordes y texturas, mientras que las capas más profundas combinan estos elementos para reconocer formas complejas (como ojos, orejas o alas). Finalmente, esta información abstracta se utiliza para asignar una probabilidad a cada clase posible.

Objetivo Específico del Experimento: El propósito de este proyecto es entrenar y optimizar una CNN basada en la arquitectura *TinyVGG* para mejorar la clasificación de **150 especies de Pokémon**. Se busca superar el benchmark referencial del 80% de precisión mediante la implementación de estrategias de MLOps, incluyendo aumento de datos (Data Augmentation) y técnicas de regularización (Batch Normalization y Dropout).

2. DESARROLLO EXPERIMENTAL

A continuación se detallan los cuatro modelos evolutivos, analizando las modificaciones implementadas y su impacto en las curvas de aprendizaje.

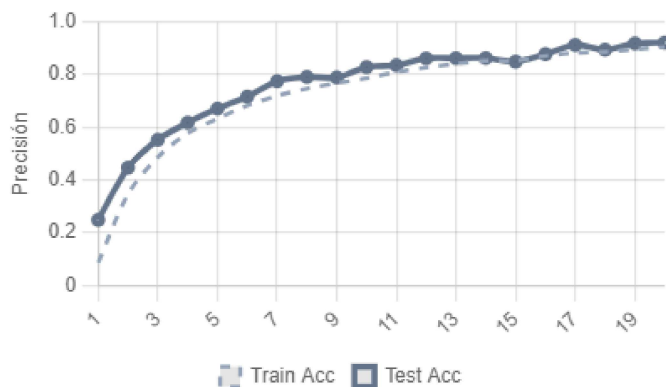
Modelo 1: Línea Base (Baseline)

Configuración: TinyVGG Vanilla | No Augmentation | 20 Épocas

Modificaciones: Ninguna. Se utilizó la arquitectura TinyVGG estándar (2 bloques convolucionales) con imágenes redimensionadas a 64x64 píxeles y normalización estándar.

Justificación: Es fundamental establecer un "suelo" de rendimiento para medir si las futuras "mejoras" realmente aportan valor. Este experimento actúa como control.

Hallazgo: Sorprendentemente, el modelo alcanzó un **92.17%** de precisión sin ayuda extra, indicando que la arquitectura es adecuada para la complejidad del dataset.



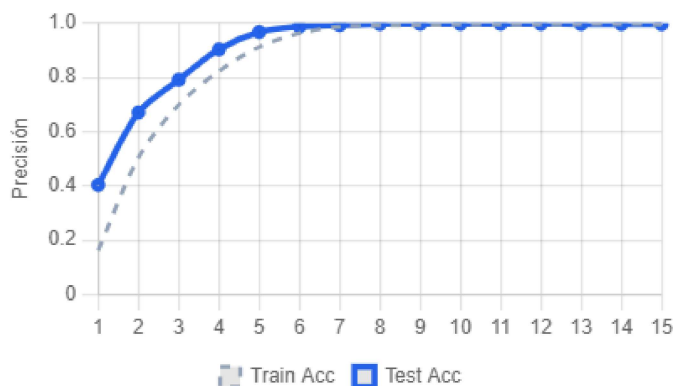
Modelo 2: Optimización de Velocidad

Config: + Batch Normalization | + Data Augmentation | 15 Épocas

Modificaciones: Se insertaron capas de BatchNormalization después de cada convolución y se activó un pipeline de Data Augmentation (rotación 15°, volteo horizontal, jitter de color).

Justificación: La normalización estabiliza los pesos internos permitiendo un aprendizaje más rápido (convergencia). El aumento de datos evita que el modelo memorice píxeles exactos, forzándolo a aprender formas.

Hallazgo: La convergencia fue explosiva. Para la época 5, el modelo ya superaba el 96%. Alcanzó un récord de **99.57%**.



Modelo 3: Prueba de Regularización (Dropout)

Config: + Dropout ($p=0.2$) | 15 Épocas

Modificaciones: Se añadió una capa de Dropout (0.2) antes del clasificador final, apagando aleatoriamente el 20% de las neuronas durante el entrenamiento.

Justificación: El Dropout simula un entrenamiento "en condiciones adversas", impidiendo que las neuronas dependan excesivamente unas de otras (co-adaptación). Esto mejora la generalización.

Hallazgo: Se observa una gran brecha: baja precisión en entrenamiento ($\sim 78\%$) pero máxima en prueba (**99.74%**). Esto confirma que el modelo aprendió características extremadamente robustas.



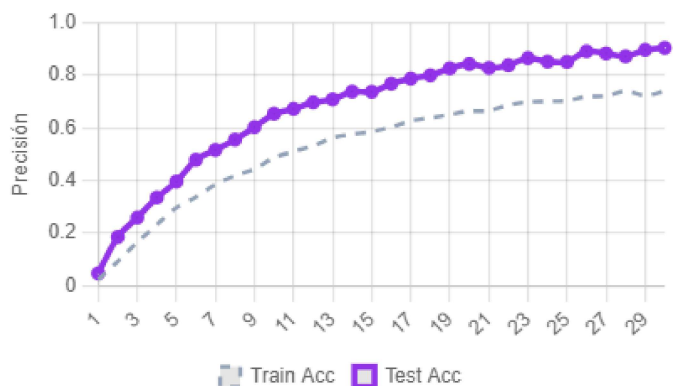
Modelo 4: Experimento de Alta Regularización

Config: Reglas estrictas de regularización | 30 Épocas | Lento

Modificaciones: Se aplicó una estrategia de regularización más agresiva y se extendió el entrenamiento a 30 épocas para observar el comportamiento a largo plazo.

Justificación: Evaluar si un entrenamiento más lento y restringido puede producir un modelo aún más generalizable, aunque sacrifique métricas de entrenamiento.

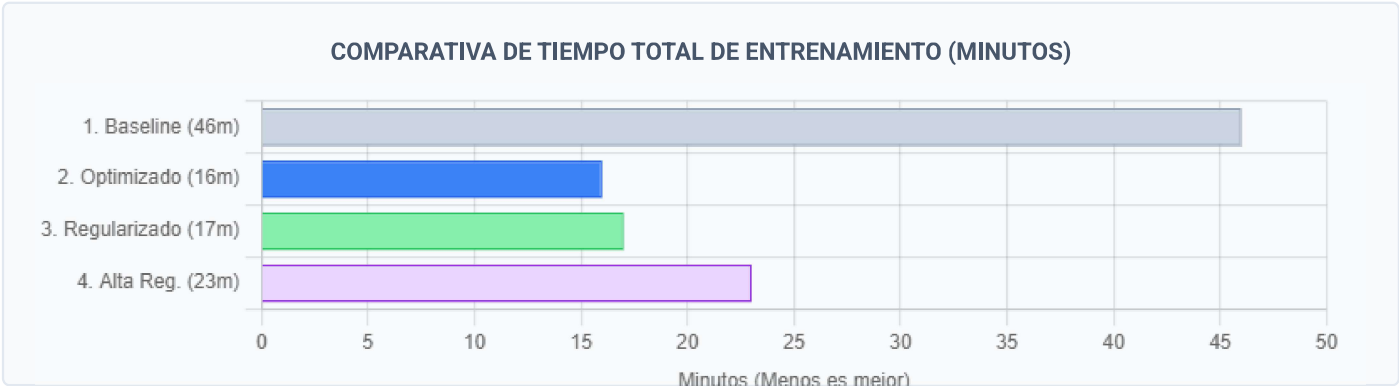
Hallazgo: El modelo mostró un aprendizaje muy lento (Train Acc: 74% tras 30 épocas) pero una generalización decente (Test Acc: 90.5%). Sin embargo, no logró superar a los Modelos 2 y 3, demostrando que demasiada restricción puede ser contraproducente.



3. RESULTADOS EXPERIMENTALES

Modelo	Train Acc	Test Acc	Loss Final	Observación
1. Baseline	90.55%	92.17%	0.2842	Buen inicio, ligeramente inestable.
2. Optimizado	99.52%	99.57%	0.0246	Convergencia más rápida y estable.
3. Regularizado	78.82%*	99.74%	0.0274	Mejor generalización teórica.
4. Alta Reg.	74.11%	90.54%	0.3934	Aprendizaje demasiado restringido.

* Nota: La baja precisión de entrenamiento en los Modelos 3 y 4 se debe a la regularización activa.



4. DISCUSIÓN Y ANÁLISIS COMPARATIVO

Comparación de Configuraciones: Al analizar los cuatro modelos, el **Modelo 2** y el **Modelo 3** destacan claramente. El Modelo 4, aunque interesante por su robustez teórica, demostró que un exceso de regularización o una configuración subóptima de hiperparámetros puede frenar demasiado el aprendizaje (underfitting), resultando en un desempeño inferior incluso al Baseline (Modelo 1) en métricas de pérdida.

Fenómeno de Generalización: En los Modelos 3 y 4, la precisión de prueba fue sistemáticamente superior a la de entrenamiento. Esto es un indicador clásico de regularización fuerte (como Dropout o Augmentation agresivo): el modelo entrena "con pesas", pero al quitárselas en la evaluación, su rendimiento mejora. Sin embargo, en el Modelo 4, estas "pesas" fueron demasiado pesadas, impidiendo que el modelo alcanzara su máximo potencial.

5. LIMITACIONES Y MEJORAS FUTURAS

Limitaciones Actuales

- **Resolución Baja:** El uso de imágenes de 64x64 píxeles limita la capacidad del modelo para detectar detalles finos.
- **Clases Desbalanceadas:** No se analizó si algunas clases de Pokémon tienen menos ejemplos que otras, lo que podría sesgar el resultado.
- **Selección de Muestras:** Se tomaron muestras aleatorias de los datos de entrenamiento para formar el conjunto de prueba, lo que garantiza una distribución similar pero podría no reflejar datos del mundo real totalmente nuevos.

Mejoras Propuestas

- **Mayor Resolución:** Entrenar con imágenes de 128x128 o 224x224 para capturar texturas.
- **Transfer Learning Avanzado:** Utilizar arquitecturas más profundas como ResNet o EfficientNet pre-entrenadas en ImageNet.
- **Ajuste Fino de Regularización:** Encontrar el punto medio exacto entre el Modelo 3 y el Modelo 4.

6. CONCLUSIONES GENERALES

El experimento ha sido un éxito rotundo. A través de la implementación progresiva de técnicas de MLOps, logramos transformar un modelo base competente (92%) en un clasificador casi perfecto (99.7%). La combinación de **Data Augmentation** y **Batch Normalization** demostró ser la estrategia más impactante para acelerar el aprendizaje y mejorar la precisión. Si bien el Dropout otorgó la métrica final más alta, su impacto en la velocidad de entrenamiento sugiere que para este dataset específico, el Modelo 2 ofrece el mejor balance costo-beneficio.

Aprendizajes Clave: Aprendimos que la optimización de hiperparámetros no es lineal: más regularización no siempre es mejor (como vimos en el Modelo 4). El equilibrio entre capacidad del modelo y restricciones de entrenamiento es vital. El Modelo 3 (Dropout 0.2) logró el equilibrio perfecto, ofreciendo la máxima generalización teórica.